清華大學
Tsinghua University

From RTL to GDS 设计流程讲解（上）

程子艺，华园，陈虹
THU-SIC
2026年5月

目录

- 综合
- 形式验证
- 布局
- 布线

综合 – 文件结构


```
DC_class/
├── Makefile          ← Make脚本，综合/综合流程封装成统一命令入口
├── designs/
│   ├── src/          ← 设计RTL源码目录
│   └── tsmc180/
│       ├── pdk/
│       │   └── config.mk          ← pdk路径变量
│       └── tinyriscv/
│           ├── config.mk          ← 综合RTL路径变量
│           ├── constraint.sdc     ← 综合时序约束文件
│           ├── constraint_for_pr.sdc  ← 后端时序约束文件
│           └── io.file            ← IO约束
├── result/
└── scripts/
    ├── fml/          ← 形式化验证脚本
    │   ├── fml_main.tcl
    │   └── fml_setup.tcl
    └── syn/          ← DC综合脚本
        ├── dc_main.tcl
        ├── dc_report.tcl
        └── dc_setup.tcl
```

综合 – 综合环境搭建

清华大学 Tsinghua University

```
#Design config
DESIGN_CONFIG ?= ./designs/tsmc180/tinyriscv/config.mk
#Process config
PDK_CONFIG ?= ./designs/tsmc180/pdk/config.mk
include $(DESIGN_CONFIG)
include $(PDK_CONFIG)
#Dir variable
export FLOW_HOME ?= .
export DESIGN_HOME  ?= $(FLOW_HOME)/designs
export SCRIPTS_DIR   ?= $(FLOW_HOME)/scripts
export RESULT_DIR ?= $(FLOW_HOME)/result
```

./Makefile

- DESIGN_CONFIG 定义了读入的Verilog文件、SDC文件、以及后端需要用到的IO文件
- PDK_CONFIG定义了工艺库中需要用到的文件：标准单元库、 LEF文件等等

综合 – DC启动命令

```
syn:
    cd $(RESULT_DIR);
    mkdir -p syn/data; \
    mkdir -p syn/log; \
    mkdir -p syn/report; \
    mkdir -p syn/work; \
    mkdir -p scanchain/data; \
    mkdir -p scanchain/report; \
    mkdir -p scanchain/log
    dc_shell -f -64 $(SCRIPTS_DIR)/syn/dc_main.tcl | tee  $(RESULT_DIR)/syn/log/syn.log
```

进入预先设置好的工作目录

创建需要的工作目录

-p 用于创建多级目录，若上级目录不存在则自动创建该目录

./makefile

- dc_shell -f -64 $(SCRIPTS_DIR)/syn/dc_main.tcl 使用64位形式启动DC运行dc_main.tcl
- tee $(RESULT_DIR)/syn/log/syn.log 将输出到命令行中的信息保存到syn.log中

每次运行后务必查看log文件，每个Warning和Error都要查看，确定是否对设计有影响

综合 – DC运行脚本 1

清華大學 Tsinghua University

```
# ------------------------------------------------------
# Set env variable
# ------------------------------------------------------
source $::env(SCRIPTS_DIR)/syn/dc_setup.tcl
```

$::env() 调用环境变量

设置运行环境

export A= abc

$::env(A) 调用环境变量

./scripts/syn/dc_main.tcl

```
define_design_lib work -path                $::env(RESULT_DIR)/syn/work
set sh_command_log_file                     $::env(RESULT_DIR)/syn/work/command.log
set_app_var alib_library_analysis_path      $::env(RESULT_DIR)/syn/work
set_svf                 $::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).svf
```

生成svf用于Formal验证

./scripts/syn/dc_setup.tcl

```
[chenh65@edaserver syn]$ cd work
[chenh65@edaserver work]$ ls
IBEX_ALU.mr                     PRIM_RAM_1P.mr                  ibex_dummy_instr-verilog.syn        prim_badbit_ram_1p-verilog.pvl
IBEX_BRANCH_PREDICT.mr          PRIM_SECDED_28_22_DEC.mr        ibex_ex_block-verilog.pvl           prim_badbit_ram_1p-verilog.syn
IBEX_COMPRESSED_DECODER.mr      PRIM_SECDED_28_22_ENC.mr        ibex_ex_block-verilog.syn           prim_clock_gating-verilog.pvl
IBEX_CONTROLLER.mr              PRIM_SECDED_39_32_DEC.mr        ibex_fetch_fifo-verilog.pvl         prim_clock_gating-verilog.syn
IBEX_CORE.mr                    PRIM_SECDED_39_32_ENC.mr        ibex_fetch_fifo-verilog.syn         prim_generic_clock_gating-verilog.pvl
IBEX_COUNTER.mr                 PRIM_SECDED_72_64_DEC.mr        ibex_icache-verilog.pvl             prim_generic_clock_gating-verilog.syn
IBEX_CSR.mr                     PRIM_SECDED_72_64_ENC.mr        ibex_icache-verilog.syn             prim_generic_ram_1p-verilog.pvl
IBEX_CS_REGISTERS.mr            PRIM_XILINX_CLOCK_GATING.mr     ibex_id_stage-verilog.pvl           prim_generic_ram_1p-verilog.syn
IBEX_DECODER.mr                 alib-52                         ibex_id_stage-verilog.syn           prim_lfsr-verilog.pvl
IBEX_DUMMY_INSTR.mr             command.log                     ibex_if_stage-verilog.pvl           prim_lfsr-verilog.syn
IBEX_EX_BLOCK.mr                ibex_alu-verilog.pvl            ibex_if_stage-verilog.syn           prim_ram_1p-verilog.pvl
IBEX_FETCH_FIFO.mr              ibex_alu-verilog.syn            ibex_load_store_unit-verilog.pvl    prim_ram_1p-verilog.syn
IBEX_ICACHE.mr                  ibex_branch_predict-verilog.pvl ibex_load_store_unit-verilog.syn    prim_secded_28_22_dec-verilog.pvl
IBEX_ID_STAGE.mr                ibex_branch_predict-verilog.syn ibex_multdiv_fast-verilog.pvl       prim_secded_28_22_dec-verilog.syn
IBEX_IF_STAGE.mr                ibex_compressed_decoder-verilog.pvl ibex_multdiv_fast-verilog.syn   prim_secded_28_22_enc-verilog.pvl
IBEX_LOAD_STORE_UNIT.mr         ibex_compressed_decoder-verilog.syn ibex_multdiv_slow-verilog.pvl   prim_secded_28_22_enc-verilog.syn
IBEX_MULTDIV_FAST.mr            ibex_controller-verilog.pvl     ibex_multdiv_slow-verilog.syn       prim_secded_39_32_dec-verilog.pvl
IBEX_MULTDIV_SLOW.mr            ibex_controller-verilog.syn     ibex_pmp-verilog.pvl                prim_secded_39_32_dec-verilog.syn
IBEX_PMP.mr                     ibex_core-verilog.pvl           ibex_pmp-verilog.syn                prim_secded_39_32_enc-verilog.pvl
IBEX_PREFETCH_BUFFER.mr         ibex_core-verilog.syn           ibex_prefetch_buffer-verilog.pvl    prim_secded_39_32_enc-verilog.syn
IBEX_REGISTER_FILE_FF.mr        ibex_counter-verilog.pvl        ibex_prefetch_buffer-verilog.syn    prim_secded_72_64_dec-verilog.pvl
IBEX_REGISTER_FILE_FPGA.mr      ibex_counter-verilog.syn        ibex_register_file_ff-verilog.pvl   prim_secded_72_64_dec-verilog.syn
IBEX_REGISTER_FILE_LATCH.mr     ibex_cs_registers-verilog.pvl   ibex_register_file_ff-verilog.syn   prim_secded_72_64_enc-verilog.pvl
IBEX_WB_STAGE.mr                ibex_cs_registers-verilog.syn   ibex_register_file_fpga-verilog.pvl prim_secded_72_64_enc-verilog.syn
PRIM_BADBIT_RAM_1P.mr           ibex_csr-verilog.pvl            ibex_register_file_fpga-verilog.syn prim_xilinx_clock_gating-verilog.pvl
PRIM_CLOCK_GATING.mr            ibex_csr-verilog.syn            ibex_register_file_latch-verilog.pvl prim_xilinx_clock_gating-verilog.syn
PRIM_GENERIC_CLOCK_GATING.mr    ibex_decoder-verilog.pvl        ibex_register_file_latch-verilog.syn
PRIM_GENERIC_RAM_1P.mr          ibex_decoder-verilog.syn        ibex_wb_stage-verilog.pvl
PRIM_LFSR.mr                    ibex_dummy_instr-verilog.pvl    ibex_wb_stage-verilog.syn
```

将DC运行产生的中间文件保存到work文件中，避免工作目录被大量中间文件弄乱

综合 – DC运行脚本 2

```
# ------------------------------------------------------------
# Lib set up
# ------------------------------------------------------------
set       target_library      $::env(DB_FILES)
set       link_library        "* $target_library"
lappend   link_library        {dw_foundation.sldb}
puts      $::env(VERILOG_FILES)
```

指定目标库

指定链接库

打印读入的Verilog文件名

在./designs/tsmc180/tinyriscv/config.mk定义

./scripts/syn/dc_main.tcl

- 配置link_library时必须包含“*”，表示DC在引用实例化模块时，首先搜索已经调进memory中的模块和单元电路

综合 – 综合RTL目录


```
export DESIGN_NICKNAME = tinyriscv
export DESIGN_NAME = tinyriscv_soc_top
export PLATFORM    = tsmc180

export VERILOG_FILES = ./designs/src/$(DESIGN_NICKNAME)/core/clint.v \
        ./designs/src/$(DESIGN_NICKNAME)/core/csr_reg.v \
        ./designs/src/$(DESIGN_NICKNAME)/core/ctrl.v \
        ...

export PR_SDC_FILE
= ./designs/$(PLATFORM)/$(DESIGN_NICKNAME)/constraint_for_pr.sdc
export SDC_FILE = ./designs/$(PLATFORM)/$(DESIGN_NICKNAME)/constraint.sdc
export IO_FILE = ./designs/$(PLATFORM)/$(DESIGN_NICKNAME)/io.file
```

项目名称

顶层名

工艺名

指定RTL

SDC路径

IO定义

./designs/tsmc180/tinyriscv/config.mk

综合 – DC运行脚本 3

```
# ------------------------------------------------------
# Design in
# ------------------------------------------------------
analyze   -format   sverilog   $::env(VERILOG_FILES)
elaborate           $::env(DESIGN_NAME)
current_design      $::env(DESIGN_NAME)
Link
check_design
```

读入Verilog

指定顶层模块

./scripts/syn/dc_main.tcl

- read_verilog *.v
- read_file –format verilog *.v
- analyze –format verilog *.v + elaborate *.v

建议使用的读入verilog方法

```
module Add#(parameter WIDTH = 4)(
   input   wire    [WIDTH-1:0]          In,
   output  reg     [WIDTH:0]            Out);
   assign Out = In + 1'b1;
endmodule
```

对于存在参数传递的Verilog模块，read_verilog和read_file命令都会使用默认的参数值，而非传递过来的参数。analyze+elaborate则会使用传递过来的参数。

www.vlsi-backend-adventure.com/logic_synthesis.html

综合 – DC运行脚本 4

```
# ------------------------------------------------------
# Design in
# ------------------------------------------------------
analyze   -format   sverilog   $::env(VERILOG_FILES)
elaborate           $::env(DESIGN_NAME)
current_design      $::env(DESIGN_NAME)
link
check_design
```

读入Verilog

指定顶层模块

./scripts/syn/dc_main.tcl

- link 命令将之前analyze和elaborate得到的各个模块、库文件，根据current_design设定的顶层模块进行组合连接，组装成完整的设计
- check_design检查目前设计，报告存在的error或warning

综合 – DC运行脚本 5

清華大學 Tsinghua University

```
# ------------------------------------------------------
# Read sdc
# ------------------------------------------------------
source $::env(SDC_FILE)
```

./scripts/syn/dc_main.tcl

```
set         clk_name            core_clock
set         clk_port_name       clk
set         clk_period          10
set         clk_io_pct          0.2

set         clk_port            [get_ports $clk_port_name]
create_clock  -name  $clk_name  -period  $clk_period  $clk_port

set  non_clock_inputs  [lsearch  -inline  -all  -not  -exact  [all_inputs]  $clk_port]
set_input_delay  [expr $clk_period * $clk_io_pct] -clock $clk_name $non_clock_inputs
set_output_delay [expr $clk_period * $clk_io_pct] -clock $clk_name [all_outputs]
```

DC中创建的时钟名字（自定义）

设计中时钟端口的名字

时钟周期

io延时占时钟周期的比例

抓取时钟端口

创建时钟

signs/tsmc180/tinyriscv/constraint.sdc

```
lsearch -all -inline -not -exact {a b c a d e a f g a} a
-> b c d e f g
```

综合 – DC运行脚本 6

```
# ------------------------------------------------------------
# Group setting
# ------------------------------------------------------------
remove_path_group –all
set reg [filter_collection [all_registers] "is_clock_gate != true"]
set input [all_inputs]
set output [all_outputs]

group_path -name reg2reg -weight 50 -critical_range 6 -from $reg -to $reg
group_path -name in2reg -weight 10 -critical_range 0.5 -from $input -to $reg
group_path -name reg2out -weight 10 -critical_range 0.5 -from $reg -to $output
```

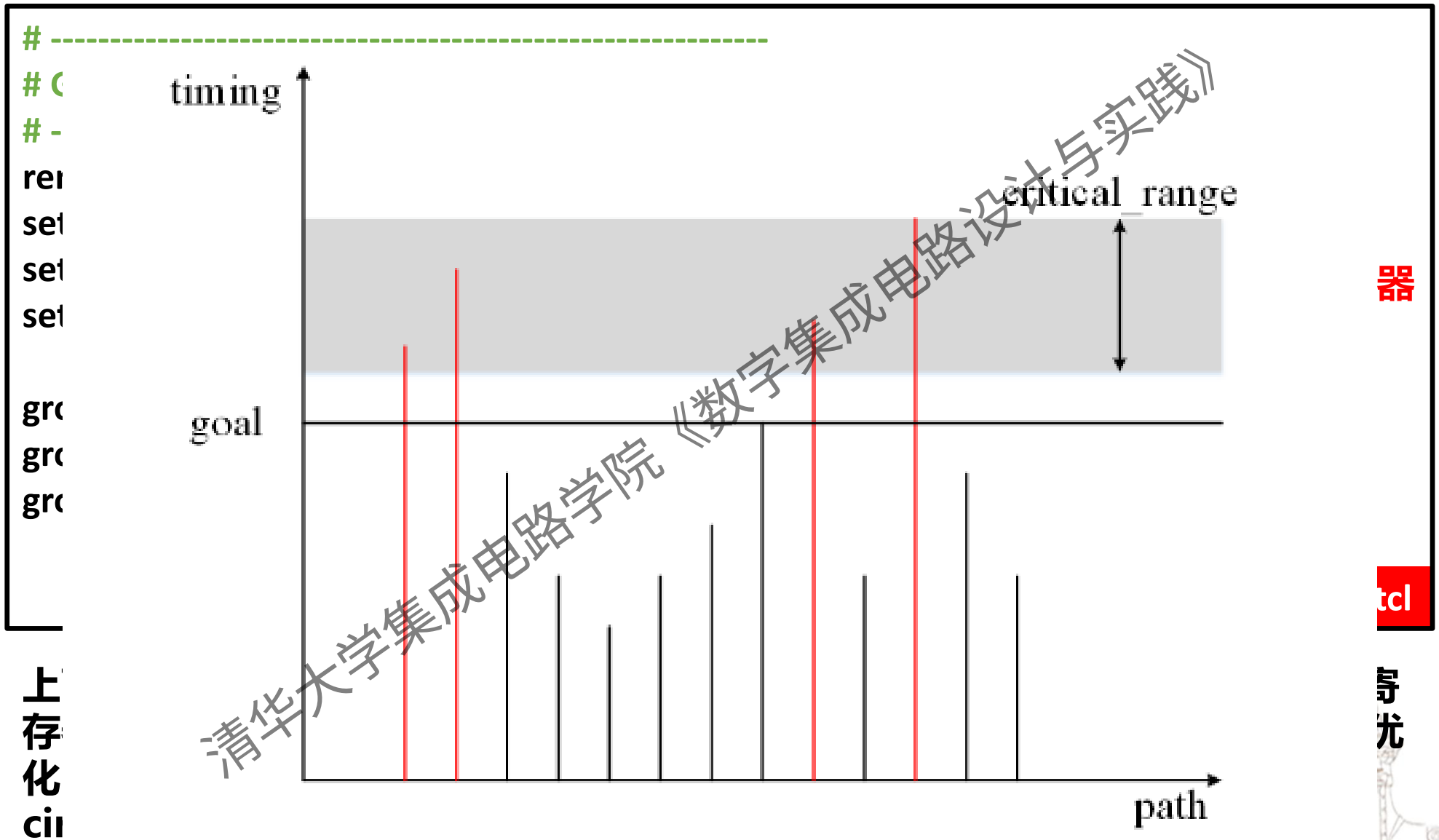
去除之前可能存在的所有group

找到所有无clock gating的寄存器

./scripts/syn/dc_main.tcl

上面的group_path命令将路径划分成三类：输入到寄存器，寄存器到寄存器，寄存器到输出。寄存器到寄存器有着更大的weight，DC会对其下更大的力度进行优化。DC默认只对一个group中的最坏路径进行优化，但是我们可以通过cirtical_range，使得DC对其余的违例路径进行优化。

综合 – DC运行脚本 7



综合 – DC运行脚本 8

```
# ------------------------------------------------------------
# Setting dontuse cell
# ------------------------------------------------------------
puts "DONTUSE“
foreach cell [split $::env(DONT_USE_CELLS)] {
   get_lib_cell  */$cell
   set_dont_use [get_lib_cell */$cell]
}
redirect $::env(RESULT_DIR)/syn/report/check_timing.rpt {check_timing}
```

./scripts/syn/dc_main.tcl

部分单元库的单元，由于工艺、性能功耗面积等原因，工艺厂商建议或者后端建议，不要使用的单元。在综合步骤设置dont_use属性，就可以避免综合使用这些单元。

综合 – DC运行脚本 9

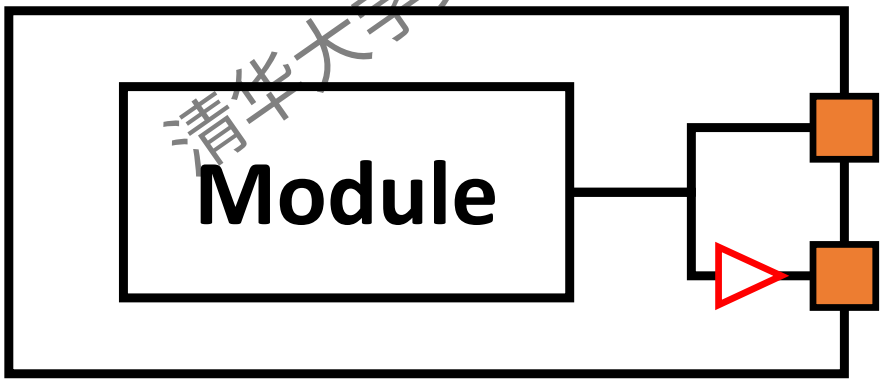
清華大學 Tsinghua University

```
# ------------------------------------------------------------
# Compile design
# ------------------------------------------------------------
set_fix_multiple_port_nets -all -buffer_constants [get_designs *]
compile_ultra -timing_high_effort_script -scan
```

./scripts/syn/dc_main.tcl

- 第一次综合
- 因为后端布局布线工具，很难读取包含tri wires ,tran 源语，assign语句的网表。因此要求我们在生成网表的时候尽可能消除assign语句。

  set_fix_multiple_port_nets 可以避免综合后的网表出现assign语句



-buffer_constants 让DC通过插入buffer的方式来解决multiple_port的问题

综合 – DC运行脚本 10


```
# ------------------------------------------------------
# Compile design incremental
# ------------------------------------------------------
compile_ultra -scan –incremental

# ------------------------------------------------------
# post-syn report & data out
# ------------------------------------------------------
source $::env(SCRIPTS_DIR)/syn/dc_report.tcl
exit
```

./scripts/syn/dc_main.tcl

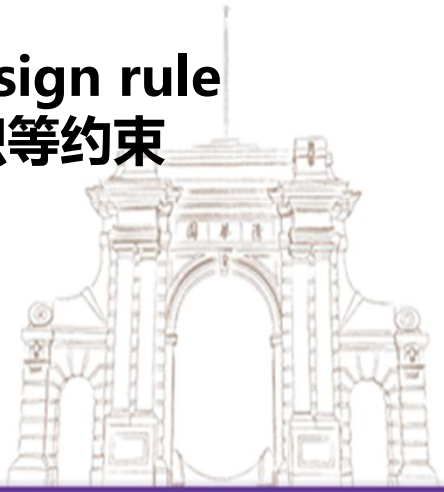
综合 – DC运行脚本 11

清华大学 Tsinghua University

```
# ------------------------------------------------------------
# output synthesis report
# ------------------------------------------------------------
redirect $::env(RESULT_DIR)/syn/report/check_design_after_syn.rpt {check_design}
report_timing -tran -net -input -max_paths 10000 -nworst 5 -nosplit -slack_lesser_than
0.0
redirect $::env(RESULT_DIR)/syn/report/qor.rpt {report_qor -nosplit}
redirect $::env(RESULT_DIR)/syn/report/violation.rpt {report_constraint -all_violators}
```

./scripts/syn/dc_report.tcl

- check_design　　检查设计中的error和warning
- report_timing　　报告违例路径
- report_qor　　报告timing-path, cell数目, 面积, Design rule
- report_constraint　　报告违反的约束，包括时序, 能耗, 面积等约束



综合 – DC运行脚本 12

```
# ------------------------------------------------------------
# output netlist for P&R
# ------------------------------------------------------------
write -format verilog -h –output
$::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).syn.v

write -format ddc -h -output
$::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).rpt.ddc

write_scan_def -output
$::env(RESULT_DIR)/scanchain/data/$::env(DESIGN_NAME).scan.def
```

./scripts/syn/dc_report.tcl

- 保存综合后生成的网表
- 保存综合工程
- 保存插入的scan chain信息

综合 – DC运行脚本 13

```
# ------------------------------------------------------------
# dont touch cell list output
# ------------------------------------------------------------
foreach cell [get_object_name [get_cells -h -f
"is_mapped == true && is_hierarchical == false && dont_touch == true"]] {
redirect -append -file $::env(RESULT_DIR)/syn/report/dont_touch.syn.tcl
{puts "set_dont_touch \[get_cells $cell\]"}
}

# ------------------------------------------------------------
# dont use cell list output
# ------------------------------------------------------------
foreach cell [get_object_name [get_lib_cells -f "dont_use == true" */*]] {
redirect -append -file $::env(RESULT_DIR)/syn/report/dont_use.syn.tcl {puts
"set_dont_use \[get_lib_cells */$cell\]"}
}
```

./scripts/syn/dc_report.tcl

- 导出 dont touch 和 dont use 信息用于后端

综合 – 报告阅读1

清华大学 Tsinghua University

```
****************************************
check_design summary:
Version:     Q-2019.12-SP5-2
Date:        Mon May  6 13:12:05 2024
****************************************

                    Name                                                Total
--------------------------------------------------------------------------------
Inputs/Outputs                                                          32
    Unconnected ports (LINT-28)                                         32

Cells                                                                   32
    Connected to power or ground (LINT-32)                              30
    Nets connected to multiple pins on same cell (LINT-33)               2

Tristate                                                                 2
    Single tristate driver drives an input pin (LINT-63)                 2
--------------------------------------------------------------------------------

Warning: In design 'rom', port 'addr_i[31]' is not connected to any nets. (LINT-28)
Warning: In design 'rom', port 'addr_i[30]' is not connected to any nets. (LINT-28)
Warning: In design 'rom', port 'addr_i[29]' is not connected to any nets. (LINT-28)
Warning: In design 'rom', port 'addr_i[28]' is not connected to any nets. (LINT-28)
Warning: In design 'rom', port 'addr_i[1]' is not connected to any nets. (LINT-28)
Warning: In design 'rom', port 'addr_i[0]' is not connected to any nets. (LINT-28)
Warning: In design 'uart_debug', port 'mem_addr_o[1]' is not connected to any nets. (LINT-28)
Warning: In design 'uart_debug', port 'mem_addr_o[0]' is not connected to any nets. (LINT-28)
Warning: In design 'uart_debug', port 'mem_rdata_i[31]' is not connected to any nets. (LINT-28)
Warning: In design 'uart_debug', port 'm
Warning: In design 'uart_debug', port 'm
```

./result/syn/report/check_design_after_syn.rpt

绝不能因为是Warning就不管，要一一确认没问题

综合 – 报告阅读2

Information: Updating design information... (UID-85)
Warning: Design 'ibex_core' contains 4 high-fanout nets. A fanout number of 1000 will be used for delay calculations involving these nets. (TIM-134)

Information: Checking generated_clocks...
Information: Checking loops...
Information: Checking no_input_delay...
Information: Checking unconstrained_endpoints...
Information: Checking pulse_clock_cell_type...
Information: Checking no_driving_cell...
Information: Checking partial_input_delay...
1

./result/syn/report/check_timing.rpt

- 我们可以用 all_high_fanout -nets -threshold 100 来报告出 high-fanout nets
- 注意上面的1000是用来计算延时的fanout number，不是真正的fanout number

综合 – 报告阅读3

清華大學 Tsinghua University

Timing Path Group 'reg2reg'	
Levels of Logic:	72.00
Critical Path Length:	9.46
Critical Path Slack:	0.00
Critical Path Clk Period:	10.00
Total Negative Slack:	0.00
No. of Violating Paths:	0.00
Worst Hold Violation:	-0.05
Total Hold Violation:	-0.17
No. of Hold Violations:	5.00

Timing Path Group 'in2reg'	
Levels of Logic:	30.00
Critical Path Length:	5.66
Critical Path Slack:	1.50
Critical Path Clk Period:	10.00
Total Negative Slack:	0.00
No. of Violating Paths:	0.00
Worst Hold Violation:	0.00
Total Hold Violation:	0.00
No. of Hold Violations:	0.00

Timing Path Group 'reg2out'	
Levels of Logic:	2.00
Critical Path Length:	0.53
Critical Path Slack:	7.17
Critical Path Clk Period:	10.00
Total Negative Slack:	0.00
No. of Violating Paths:	0.00
Worst Hold Violation:	0.00
Total Hold Violation:	0.00
No. of Hold Violations:	0.00

Design Rules	
Total Number of Nets:	39111
Nets With Violations:	1
Max Trans Violations:	0
Max Cap Violations:	1

./result/syn/report/qor.rpt

综合 – 报告阅读3


```
min_delay/hold ('reg2reg' group)

                          Required         Actual
Endpoint                Path Delay     Path Delay          Slack
-------------------------------------------------------------------
u_jtag_top/u_jtag_driver/ir_reg_reg[1]/D
                             0.37          0.33 r          -0.05   (VIOLATED)
u_jtag_top/u_jtag_driver/ir_reg_reg[2]/D
                             0.37          0.33 r          -0.05   (VIOLATED)
u_jtag_top/u_jtag_driver/ir_reg_reg[3]/D
                             0.37          0.33 r          -0.04   (VIOLATED)
u_jtag_top/u_jtag_driver/ir_reg_reg[4]/D
                             0.37          0.35 r          -0.02   (VIOLATED)
spi_0/clk_cnt_reg[0]/D       0.25          0.24 r          -0.01   (VIOLATED)
```

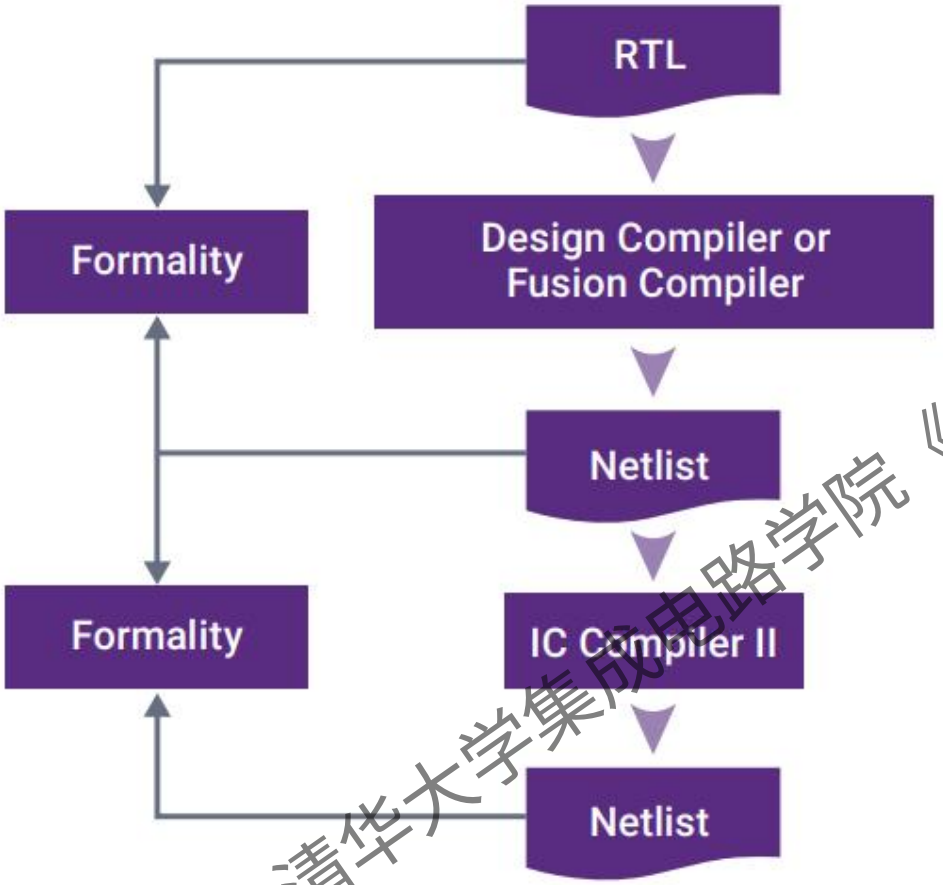
./result/syn/report/ violation.rpt

- 我们可以提高n3586驱动门的驱动能力来解决该vio
- 建议不处理，交由后端进行处理。在数值如此小的情况下，在前端处理往往会过修复，影响到性能

目录

- 综合
- **形式验证**
- 布局
- 布线

形式验证 – 背景



- 简单来说，Formality 是用来判断设计的两个版本在功能上是否一致的工具
- 在 Formality 中，存在一个重要的概念：容器（container）
- Formality 通过比较 reference 和 implementation container 中的设计，判断两个 container 中的设计是否在功能上等价

形式验证 – FM启动命令


```
fml_syn:
        cd $(RESULT_DIR); mkdir -p fml_syn/data; \
                mkdir -p fml_syn/log; \
                mkdir -p fml_syn/report; \
                mkdir -p fml_syn/work
        fm_shell -64 -f $(SCRIPTS_DIR)/fml/fml_main.tcl \
                -work_path $(RESULT_DIR)/fml_syn/work \
                | tee $(RESULT_DIR)/fml_syn/report/fml.log
```

./Makefile

- fm_shell -f -64 $(SCRIPTS_DIR)/fml/fml_main.tcl 使用64位形式启动FM运行fml_main.tcl
- tee $(RESULT_DIR)/fml_syn/report/fml.log 将输出到命令行中的信息保存到fml.log中

每次运行后务必查看log文件，每个Warning和Error都要查看，确定是否对设计有影响

形式验证 – FM运行脚本1


```
# ------------------------------------------------------------
# fml variable setting
# ------------------------------------------------------------
source     $::env(SCRIPTS_DIR)/fml/fml_setup.tcl
```

设置运行环境

./scripts/fml/fml_main.tcl

```
set sh_command_log_file "$::env(RESULT_DIR)/fml_syn/work/fm_shell_command.log
set formality_log_name “ $::env(RESULT_DIR)/fml_syn/work/formality.log“
define_design_lib -r -path "$::env(RESULT_DIR)/fml_syn/work" work
set verification_set_undriven_signals 0
set hdlin_vhdl_auto_file_order false
```

./scripts/fml/fml_setup.tcl

- 将FM运行产生的中间文件保存到work文件中，避免工作目录被大量中间文件弄乱
- set verification_set_undriven_signals 0将没有驱动的信号线数值设置为0，避免比较时出现不同
- set hdlin_vhdl_auto_file_order false 让FM读入设计时不自动排序

形式验证 – FM运行脚本2

```
# ------------------------------------------------------
# Read db
# ------------------------------------------------------
read_db   -technology_library   $::env(DB_FILES)
# ------------------------------------------------------
# Svf In
# ------------------------------------------------------
set_svf    $::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).svf
```

读入库文件

读入Automated setup file

./scripts/fml/fml_main.tcl

- 读入库文件，这些库文件会被同时用于 reference 和 implementation design中
- 读入Automated setup file（.svf文件，综合时生成）。svf文件记录了优化过程中的各项逻辑操作，比如replace, merge, uniquify, retime, reg_constant等等，用于辅助formality verification。

形式验证 – FM运行脚本3

```
# ------------------------------------------------------------
# Reference design
# ------------------------------------------------------------
foreach file $::env(VERILOG_FILES) {
  read_verilog -r $file
}
set_top r:WORK/$::env(DESIGN_NAME)
# ------------------------------------------------------------
# Implement design
# ------------------------------------------------------------
read_verilog -i $::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).syn.v
set_top i:WORK/$::env(DESIGN_NAME)
```

./scripts/fml/fml_main.tcl

注意：
-r r: 指定Reference container
-i i: 指定Implementation container

- 读入 Reference design 到 Reference container 中
- 读入 Implement design 到 Implementation container 中

形式验证 – FM运行脚本4

```
# ------------------------------------------------------------
# Run match
# ------------------------------------------------------------
match
report_matched_points > $::env(RESULT_DIR)/fml_syn/report/matched_points.rpt
report_unmatched_points > $::env(RESULT_DIR)/fml_syn/report/unmatched_points.rpt
report_unmatched_points -status unread >$::env(RESULT_DIR)/fml_syn/report/unread.rpt
```

FM主要的命令，根据比较点将reference和implement design对应起来

./scripts/fml/fml_main.tcl

- report_matched_points　　输出所有的matched points
- report_unmatched_points　　输出所有的unmatched points
- report_unmatched_points -status unread　　输出所有的unread points

unread points通常是设计中没有用到的寄存器，例如说你设计了一个加法器并将结果保存到寄存器中，但是后面只用了最高的几位，低几位没用到。综合时这几位寄存器可能会被优化掉，这样在对比时就会出现unread points。

形式验证 – FM运行脚本5

清华大学 Tsinghua University

```
# ------------------------------------------------------------
# Run verify
# ------------------------------------------------------------
verify
```

← FM主要的命令，判断二者的逻辑功能是否等同

```
report_failing_points > $::env(RESULT_DIR)/fml_syn/report/fail_points.rpt
report_aborted_points > $::env(RESULT_DIR)/fml_syn/report/aborted.rpt
save_session -replace $::env(RESULT_DIR)/fml_syn/data/rtl2syn.fss
```

./scripts/fml/fml_main.tcl

- report_failing_points　　输出所有的failing points
- report_aborted_points　　输出所有的aborted points

failing points是设计中逻辑功能不等价的点
aborted points是FM无法判断逻辑功能是否等价的点（一般出现在组合逻辑环路）

形式验证 – FM报告1

清華大學 Tsinghua University

No aborted compare points.
1

./result/fml_syn/report/aborted.rpt

No failing compare points.
1

./result/fml_syn/report/fail_points.rpt